

Block 1 Problem Set: Number Theory and Algebraic Foundations

Key idea: computation in finite arithmetic structures.

Unless otherwise stated, all algorithms must be written from first principles. Built-in routines may be used for testing, but not as the implementation being tested. For software problems, submit both source code and a `pytest` test suite.

1 Proof and Calculation Problems

P1. The division algorithm and uniqueness. Let $a \in \mathbb{Z}$ and let $n \in \mathbb{Z}$ with $n > 0$. Prove that there exist unique integers q, r such that

$$a = qn + r, \quad 0 \leq r < n.$$

Your proof should establish both existence and uniqueness explicitly.

P2. Basic divisibility. Suppose $a, b, c \in \mathbb{Z}$.

- (a) Prove that if $a \mid b$ and $b \mid c$, then $a \mid c$.
- (b) Prove that if $a \mid b$ and $a \mid c$, then $a \mid (rb + sc)$ for all $r, s \in \mathbb{Z}$.
- (c) Determine all integers d satisfying $d \mid 84$ and $d \mid 126$.
- (d) Verify directly that the largest positive such d is 42.

P3. Euclid's lemma. Let p be prime. Prove that if $p \mid ab$, then $p \mid a$ or $p \mid b$. You may use Bézout's identity, but not unique factorization.

P4. Correctness of the Euclidean algorithm. Prove that for integers a, b with $b \neq 0$,

$$\gcd(a, b) = \gcd(b, a \bmod b).$$

Use this identity to prove that the Euclidean algorithm terminates and returns $\gcd(a, b)$.

P5. Extended Euclidean algorithm by hand. For each pair below, compute $\gcd(a, b)$ and integers x, y such that $ax + by = \gcd(a, b)$:

$$(252, 198), \quad (65537, 12345), \quad (391, 299).$$

Show the Euclidean remainder sequence and the back-substitution.

P6. Bézout coefficients and inverses. Prove that a has a multiplicative inverse modulo n if and only if $\gcd(a, n) = 1$. Then compute, if they exist,

$$17^{-1} \pmod{3120}, \quad 42^{-1} \pmod{2017}, \quad 39^{-1} \pmod{105}.$$

P7. Linear congruences. Let $a, b, n \in \mathbb{Z}$ with $n > 0$ and $d = \gcd(a, n)$. Prove that

$$ax \equiv b \pmod{n}$$

has a solution if and only if $d \mid b$. Prove further that, when a solution exists, there are exactly d incongruent solutions modulo n . Apply your result to solve:

$$18x \equiv 30 \pmod{42}, \quad 35x \equiv 10 \pmod{50}, \quad 14x \equiv 9 \pmod{35}.$$

P8. Equivalence classes and well-defined operations. Define $a \sim b$ if $a \equiv b \pmod{n}$.

- (a) Prove that $\sim$ is an equivalence relation.
- (b) Prove that addition and multiplication on residue classes,

$$[a] + [b] = [a + b], \quad [a][b] = [ab],$$

are well defined.

- (c) Explain precisely why division is not always well defined in $\mathbb{Z}/n\mathbb{Z}$.

P9. Unique factorization and gcd/lcm. Assuming the Fundamental Theorem of Arithmetic, let

$$a = \prod_p p^{\alpha_p}, \quad b = \prod_p p^{\beta_p},$$

where all but finitely many exponents are zero. Prove

$$\gcd(a, b) = \prod_p p^{\min(\alpha_p, \beta_p)}, \quad \text{lcm}(a, b) = \prod_p p^{\max(\alpha_p, \beta_p)},$$

and deduce

$$\gcd(a, b) \text{lcm}(a, b) = |ab|.$$

P10. p -adic valuations. For a prime p and nonzero integer n , define $v_p(n)$ to be the exponent of p in the prime factorization of n . Prove

$$v_p(ab) = v_p(a) + v_p(b)$$

and

$$v_p(a + b) \geq \min\{v_p(a), v_p(b)\}.$$

Give an example where the second inequality is strict.

P11. Euler's phi function.

- (a) Prove that if p is prime, then $\varphi(p) = p - 1$.
- (b) Prove that $\varphi(p^k) = p^k - p^{k-1}$.
- (c) Prove that if $\gcd(m, n) = 1$, then

$$\varphi(mn) = \varphi(m)\varphi(n).$$

- (d) Compute $\varphi(360)$, $\varphi(1001)$, and $\varphi(2^{10}3^4)$.

P12. Fermat and Euler. Using group-theoretic language, prove Euler's theorem: if $\gcd(a, n) = 1$, then

$$a^{\varphi(n)} \equiv 1 \pmod{n}.$$

Deduce Fermat's little theorem as a special case.

P13. Multiplicative order. For $a \in (\mathbb{Z}/n\mathbb{Z})^\times$, define $\text{ord}_n(a)$ to be the least positive r such that $a^r \equiv 1 \pmod{n}$.

- (a) Prove that $\text{ord}_n(a) \mid \varphi(n)$.
- (b) Compute $\text{ord}_{17}(2)$, $\text{ord}_{17}(4)$, and $\text{ord}_{29}(2)$.
- (c) If $\text{ord}_n(a) = r$, prove that

$$a^u \equiv a^v \pmod{n} \iff u \equiv v \pmod{r}.$$

P14. Square-and-multiply. Let $e = \sum_{i=0}^t e_i 2^i$ be the binary expansion of e . Derive a square-and-multiply algorithm for computing $a^e \pmod{n}$. Prove its correctness using a loop invariant, and show that it requires $O(\log e)$ modular multiplications. Then compute

$$7^{560} \pmod{561}$$

using repeated squaring, showing the intermediate powers you actually use.

P15. Chinese remainder theorem: two moduli. Let $\gcd(m, n) = 1$. Prove that the map

$$\psi : \mathbb{Z}/mn\mathbb{Z} \longrightarrow \mathbb{Z}/m\mathbb{Z} \times \mathbb{Z}/n\mathbb{Z}, \quad [x]_{mn} \mapsto ([x]_m, [x]_n)$$

is a ring isomorphism. Your proof must include injectivity and surjectivity.

P16. Constructive CRT. Solve the system

$$x \equiv 4 \pmod{7}, \quad x \equiv 8 \pmod{11}, \quad x \equiv 3 \pmod{13}.$$

Construct the solution using explicit CRT coefficients rather than search.

P17. CRT and units. For distinct primes p, q , use the CRT isomorphism to prove

$$(\mathbb{Z}/pq\mathbb{Z})^\times \cong \mathbb{F}_p^\times \times \mathbb{F}_q^\times.$$

Deduce

$$\varphi(pq) = (p-1)(q-1).$$

P18. Groups, rings, and fields. For each structure below, determine whether it is a group under addition, a group under multiplication, a ring, and/or a field. Justify every answer.

$$\mathbb{Z}/8\mathbb{Z}, \quad (\mathbb{Z}/8\mathbb{Z})^\times, \quad \mathbb{Z}/7\mathbb{Z}, \quad 2\mathbb{Z}, \quad \mathbb{F}_5[x]/(x^2 + 2).$$

P19. Irreducibility over finite fields.

- (a) Determine whether $x^2 + x + 1$ is irreducible over $\mathbb{F}_2$.
- (b) Determine whether $x^3 + x + 1$ is irreducible over $\mathbb{F}_2$.
- (c) Determine whether $x^2 + 1$ is irreducible over $\mathbb{F}_5$.

- (d) Prove that a polynomial of degree 2 or 3 over a field is reducible if and only if it has a root in that field.

P20. Arithmetic in an extension field. Let

$$K = \mathbb{F}_2[x]/(x^3 + x + 1),$$

and write $\alpha = [x]$.

- (a) List all elements of K .
 (b) Reduce $\alpha^3, \alpha^4, \dots, \alpha^7$ to the basis $\{1, \alpha, \alpha^2\}$.
 (c) Compute

$$(\alpha^2 + \alpha + 1)(\alpha^2 + 1).$$

- (d) Find the inverse of $\alpha^2 + \alpha + 1$.
 (e) Determine the multiplicative order of α .

P21. Primitive roots in a prime field. Let p be prime and let $g \in \mathbb{F}_p^\times$. Prove that g is a generator of $\mathbb{F}_p^\times$ if and only if

$$g^{(p-1)/\ell} \not\equiv 1 \pmod{p}$$

for every prime divisor ℓ of $p - 1$. Use this criterion to determine whether 2, 3, and 5 are primitive roots modulo 23.

P22. Optional enrichment: quadratic residues. For odd prime p , call $a \not\equiv 0 \pmod{p}$ a quadratic residue if $x^2 \equiv a \pmod{p}$ has a solution.

- (a) Prove that exactly $(p - 1)/2$ nonzero residue classes are quadratic residues.
 (b) Compute the quadratic residues modulo 11 and modulo 19.
 (c) If g generates $\mathbb{F}_p^\times$, characterize the quadratic residues as powers of g .

2 Software Implementation Problems

S1. Euclidean and extended Euclidean algorithms.

Implement:

```
def gcd(a: int, b: int) -> int:
    ...

def xgcd(a: int, b: int) -> tuple[int, int, int]:
    """Return (g, x, y) with g >= 0 and ax + by = g = gcd(a,b)."""
```

Requirements:

- Do not call `math.gcd` inside either implementation.
- Handle negative inputs and zero correctly.
- `xgcd(0,0)` must return a documented, consistent result.
- Include property-style tests on at least 1000 pseudorandom input pairs.

At minimum, your `pytest` suite must verify:

$$g \mid a, \quad g \mid b, \quad ax + by = g, \quad g = \gcd(|a|, |b|).$$

Benchmark the number of Euclidean remainder steps for consecutive Fibonacci pairs (F_{k+1}, F_k) and for uniformly sampled pairs of comparable bit length. Report and explain the observed difference.

S2. Modular inversion and linear congruences.

Implement:

```
class NonInvertibleError(ValueError):
    pass

def invmod(a: int, n: int) -> int:
    ...

def solve_linear_congruence(a: int, b: int, n: int) -> list[int]:
    """Return all incongruent solutions x in range(n), sorted."""
```

Requirements:

- `invmod` must raise `NonInvertibleError` if $\gcd(a, n) \neq 1$.
- `solve_linear_congruence` must return every solution and no duplicates.
- Do not solve by checking every $x \in \{0, \dots, n-1\}$.

Your tests must include invertible and noninvertible cases, negative a , and cases with 0, 1, and more than one solution.

S3. Square-and-multiply modular exponentiation.

Implement:

```
def powmod(base: int, exponent: int, modulus: int) -> int:
    ...
```

Requirements:

- Use square-and-multiply; do not call Python's three-argument `pow` in the implementation.
- Require `exponent` ≥ 0 and `modulus` ≥ 2 .
- Test against `pow(base, exponent, modulus)` on at least 5000 random cases.

Benchmark `powmod` against a naïve repeated-multiplication implementation for exponents with bit lengths 8, 16, 32, 64, 128, 256. Plot or tabulate runtime against exponent bit length and explain the asymptotic behavior.

S4. Constructive Chinese remainder theorem.

Implement:

```
def crt(residues: list[int], moduli: list[int]) -> tuple[int, int]:
    """Return (x, M) with  $0 \leq x < M$  and  $x$  congruent to  $\text{residues}[i]$ 
    modulo  $\text{moduli}[i]$  for every  $i$ . Assume pairwise-coprime moduli."""
```

Requirements:

- Verify pairwise coprimality and raise a clear exception if it fails.
- Construct the solution from modular inverses; do not search through candidates.
- Include randomized tests that generate a hidden x , reduce it modulo random pairwise-coprime moduli, reconstruct it, and verify uniqueness modulo the product.

As a timing experiment, compare reconstruction from $k = 2, 4, 8, 16, 32$ randomly generated small prime moduli.

S5. Prime-field arithmetic.

Create a class or immutable data type representing $\mathbb{F}_p$ for prime p . It must support:

$+$, $-$, $\times$, $/$, negation, equality, and exponentiation.

All operations must reduce automatically modulo p .

Your tests must verify the field axioms on randomly sampled triples of elements, including associativity, distributivity, identities, and multiplicative inverses for nonzero elements. Your constructor must reject at least several obvious composite values of p .

S6. Polynomial arithmetic and a small extension field.

Represent polynomials over $\mathbb{F}_p$ and implement:

```
def poly_add(f, g, p):  
    ...  
  
def poly_mul(f, g, p):  
    ...  
  
def poly_divmod(f, g, p):  
    ...  
  
def poly_mod(f, modulus_poly, p):  
    ...
```

Then implement arithmetic in

$$\mathbb{F}_{2^3} \cong \mathbb{F}_2[x]/(x^3 + x + 1).$$

Your tests must verify:

- polynomial division satisfies $f = qg + r$ with $\deg r < \deg g$;
- every nonzero element of $\mathbb{F}_{2^3}$ has an inverse;
- the multiplicative group has exactly 7 elements;
- for every nonzero a , $a^7 = 1$.

Do not hard-code a multiplication table.

S7. Orders and primitive roots.

Implement:

```
def multiplicative_order(a: int, p: int) -> int:
    ...

def primitive_roots(p: int) -> list[int]:
    ...
```

For this problem, p is an odd prime small enough that factoring $p - 1$ by trial division is acceptable.

Requirements:

- Use the prime-factor criterion from Problem P21 rather than testing every exponent $1, \dots, p - 1$ for every candidate.
- Verify that the number of primitive roots returned is $\varphi(p - 1)$.
- Test for at least 20 primes below 500.

Measure the speedup of the prime-factor criterion against a naïve order computation as p grows through the primes below 5000.

3 Attack and Codebreaking Problems

For these problems, submit the recovered plaintext and key, together with the shortest mathematical or computational justification needed to make the recovery reproducible.

A1. Affine cipher.

Alphabet: $A = 0, \dots, Z = 25$.

Ciphertext:

QCCZ IZ ZRC ALX UZAVC NPWXMZ IZ QWXVWMRZ. NPWVM ZRC VAZCNAAG IVX ZRC UQILL
NPIUU GCY. WH ZRC PAIX WU NLASGCX, PCZEPV ZA ZRC UZIZWAV IVX OIWZ HAP ZRC
UCSAVX QCUUIMC.

Recover the plaintext and affine key.

A2. Known-plaintext multiplicative cipher.

Alphabet: $A = 0, \dots, Z = 25$.

Ciphertext:

DXC BAOSAQC EW XEVVCN KNVCP DXC DXEPV JZUUP WDAEPOAWC. KWC DXC CAWD CNDPANOC
AJDCP WKNWCD.

The plaintext begins with THE.

Recover the key and plaintext.

A3. Noninvertible affine key.

Alphabet: $A = 0, \dots, Z = 25$.

The encryption rule is

$$E(x) \equiv 6x + 11 \pmod{26}.$$

Determine exactly what goes wrong. Give two distinct plaintext letters that encrypt to the same ciphertext letter, and characterize all collisions.

A4. Monoalphabetic substitution by frequency analysis.

Ciphertext:

ZIT LZXRN GY DGRXSQK QKOZIDTZOE WTUOFL VOZI Q LODHST GWLTKCQZOGF: VITF
 OFZTUTKL QKT KTRXETR DGRXSG Q YOBTR FXDWTk, QRROZOGF QFR DXSZOHSEQZOGF
 KTDQOF VTSS RTYOFTR. ZIOL LDQSS ORTQ LXHHGKZL DXEI GY DGRTKF EKNHZGUKQHIN.
 QF QZZQEATK VIG XFRTKLZQFRL ZIT QSUTWKQ EQF GYZTF KTEGUFOMT LZKXEZXKT ZIQZ
 OL OFCOLOWST QZ ZIT STCTS GY KQV LNDWGS L. YKTJXTFEN QFQSNLLOL OL GFT TQKSN
 TBQDHST. EGDDGF STZZTKL, KTHTQZTR HQZZTKFL, QFR LIGKZ VGKRL STQA OFYGKDQZOGF
 QWGXZ Q DGFGQSHIQWTZOE LXWLZOZXZOGF. ZIT SGFUTK ZIT DTLLQUT, ZIT DGKT KTSOQWST
 ZITLT LZQZOLZOEQS ESXTL WTEGDT. Q EQKTYXS EGRTWKTQATK EGDWOFTL EGXFZL VOZI
 SQFUXQUT LZKXEZXKT KQZITK ZIQF ZKXLZOFU QFN LOFUST ITXKOLZOE.

Recover the plaintext and substitution alphabet.

A5. CRT reconstruction.

A secret integer x satisfies

$$0 \leq x < 101 \cdot 103 \cdot 107$$

and

$$x \equiv 71 \pmod{101}, \quad x \equiv 38 \pmod{103}, \quad x \equiv 8 \pmod{107}.$$

Recover x .

4 Suggested Integrative Assignment

Prove the correctness of both the extended Euclidean algorithm and the constructive Chinese remainder theorem. Then annotate your implementations from Problems S1 and S4 so that each major step of the code is explicitly linked to a corresponding statement in the proof.

Your submission should make clear which facts are mathematical invariants, which are representation choices, and which conditions are necessary for failure handling.